hashlock.

Security Audit

NLO (Vault)

July 2026

info@hashlock.com.au

Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE FINDINGS ARE RESOLVED OR FORMALLY ACKNOWLEDGED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

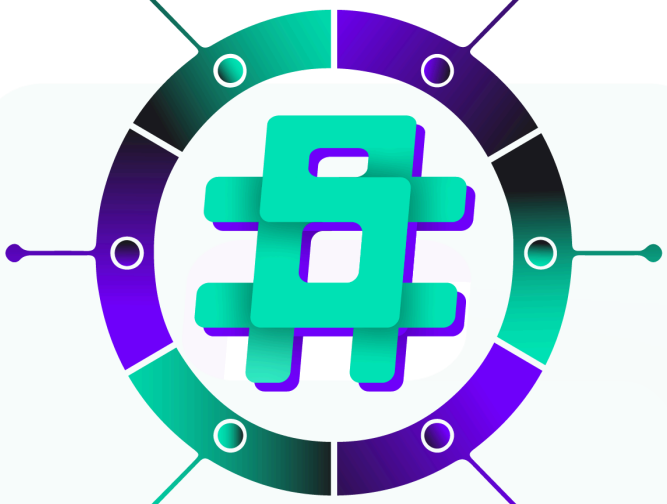
The NLO team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

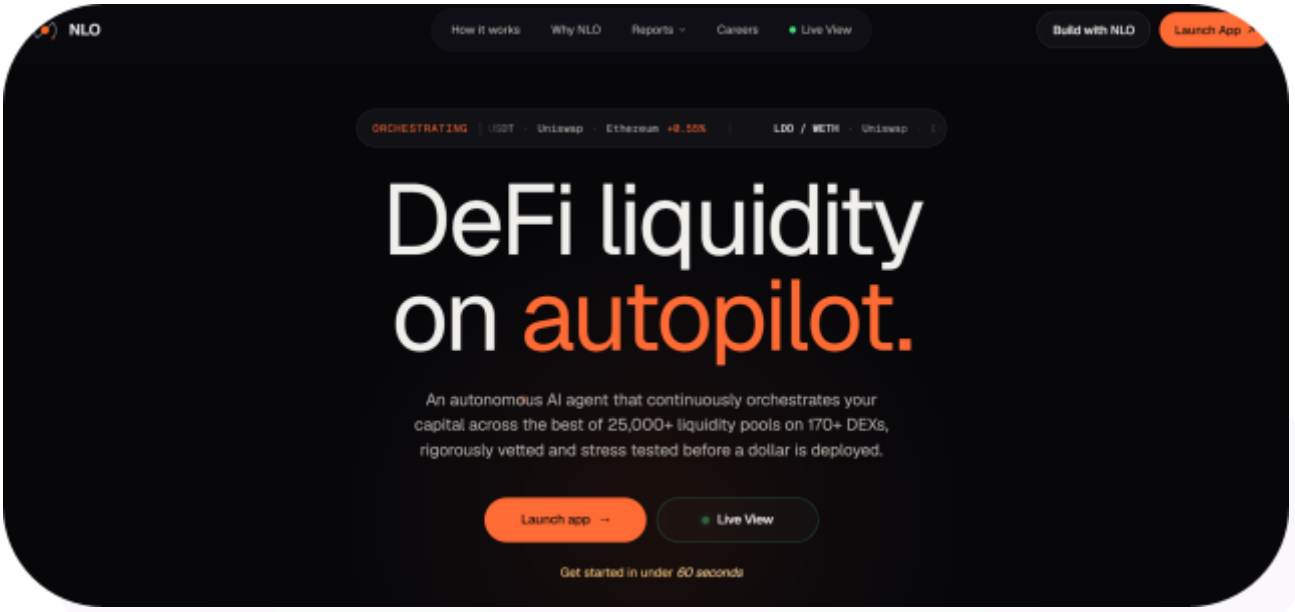
NLO is a custodial, multi-chain yield vault: users deposit a stablecoin and receive shares representing their claim on the vault's NAV, and the operator deploys the pooled capital into external DeFi strategies (DEX liquidity and swaps) to generate yield. Deposits and withdrawals are routed across six EVM chains via deBridge, with redemptions settled by the backend rather than a direct on-chain withdrawal.

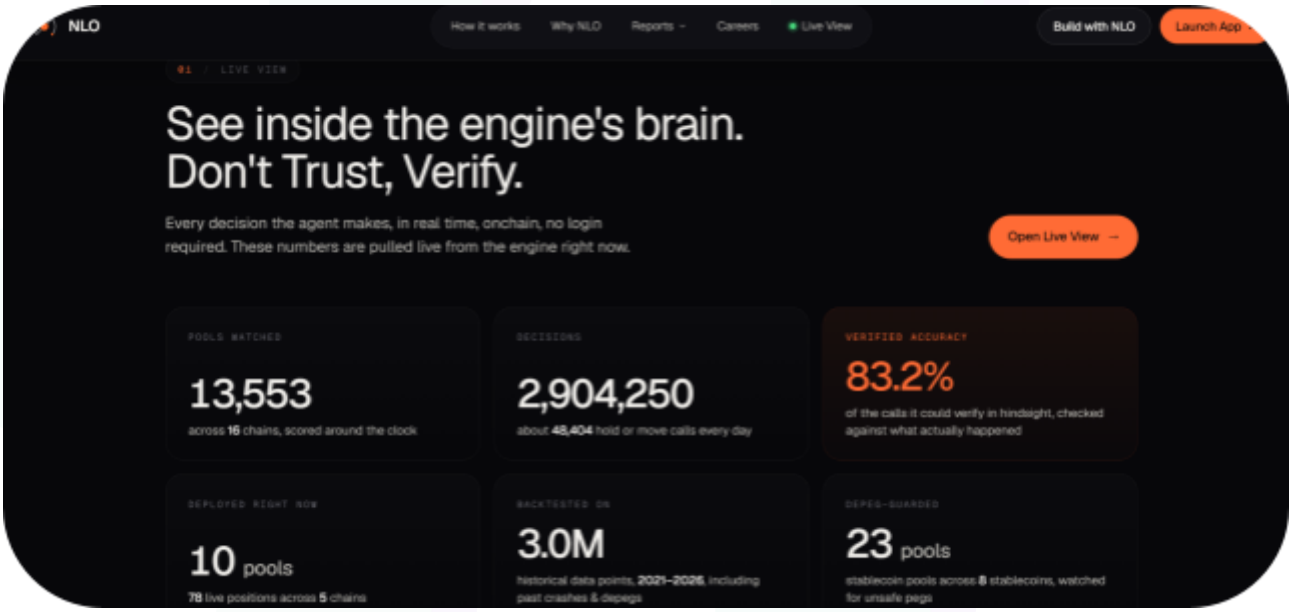
Project Name: NLO
Project Type: Vault, DeFi
Compiler Version: ^0.8.24
Website: https://nlo.finance/
Logo:

Visualised Context:



Project Visuals:





Audit Scope

We at Hashlock audited the solidity code within the NLO project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	**NLO Smart Contracts**
Network	**BSC, Ethereum, Arbitrum, Optimism,Base, Polygon and L1X**
Language	**Solidity**
Audit Date	**July, 2026**
Contract 1	CapitalSafeL1XEscrow.sol
Contract 2	L1X_CapitalSafe.sol
Contract 3	NLOIntakeRouter.sol
Contract 4	NLOPayoutRouter.sol
Contract 5	NLOVault.sol
Contract 6	WhitelistRegistry.sol
Contract 7	XUSD.sol
Contract 8	DeBridgeDecoder.sol
Audited GitHub Commit Hash	a1f2182b2b1461ef9667943c8a0e272e089be109
Fix Review GitHub Commit Hash	f0a8e91946595a4318d2e2bc86ea9885395b7fc6

Note: *Hashlock initially audited the code associated with the "Audited GitHub Commit Hash" and the findings presented in this report pertain specifically to that commit. For the "Fix Review GitHub Commit Hash", Hashlock solely verified the status of the identified findings and did not conduct a comprehensive review to assess any additional code implementations.*

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts are clearly written and well-ordered, with detailed NatSpec comments, though the protocol's logic is non-trivial spanning a custodial multi-chain vault, cross-chain intake/payout routing via deBridge, and DEX integrations. They use a series of interfaces and standard OpenZeppelin contracts.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved or acknowledged.

Hashlock found:

2 High severity vulnerabilities

2 Medium severity vulnerabilities

5 Low severity vulnerabilities

2 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
NLOVault.sol • Accepts stablecoin (USDT) deposits and mints non-transferable shares representing a claim on NAV. • Operator deploys idle capital into whitelisted DEX/LP targets via execute() to earn yield. • Accrues management and performance fees. • Redemptions settle via operator-gated payOutAndBurn() (no direct user withdrawal).	**Contract achieves this functionality.**
NLOIntakeRouter.sol • Normalizes any supported token to the canonical stablecoin and buffers it. • Bridges buffered deposits cross-chain via deBridge and credits inbound deposits. • Refunds timed-out bridges to the recorded owner.	**Contract achieves this functionality.**
NLOPayoutRouter.sol • Operator settles a payout intent; anyone can claim(), with funds sent only to the recorded owner. • Owner can emergencyRescue() stuck payouts. • Non-upgradeable; deployed at identical CREATE2 addresses across chains.	**Contract achieves this functionality.**
XUSD.sol • 18-decimal ERC-20 on L1X, paired with WL1X in a DEX pool. • Owner can mint, burn, pause, and blocklist (USDT/USDC-style issuer model); renounceOwnership is disabled.	**Contract achieves this functionality.**

L1X_CapitalSafe.sol • Extends the base vault with multi-hop exactInput swap support for the L1X capital-safe flow.	**Contract achieves this functionality.**
CapitalSafeL1XEscrow.sol • Receives the canonical-token slice (the L1X 10% portion) and, via depositFor(), atomically forwards it to an immutable recipient. • Tracks nominal 1:1 share accounting per receiver; owner can sweep stray tokens.	**Contract achieves this functionality.**
WhitelistRegistry.sol • Maintains the admin-managed allowlist of external call targets/spenders the vault and routers may interact with. • Exposes isWhitelisted() checks; admin adds/removes entries via a two-step admin-transfer pattern.	**Contract achieves this functionality.**
DeBridgeDecoder.sol • Decodes the cross-chain receiver (receiverDst) from a deBridge OrderCreation calldata so callers can validate the bridge destination.	**Contract achieves this functionality.**

Code Quality

This audit scope involves the smart contracts of the NLO project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the NLO project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contract infrastructure are based on well-known, industry-standard open-source projects (primarily OpenZeppelin). Beyond these libraries, the protocol's contracts make external calls into third-party systems notably deBridge for cross-chain routing, DEX routers/aggregators for swaps, and Uniswap-style position managers for liquidity.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-severity issues should be resolved before deployment. While they do not typically lead to a complete loss of funds, they may result in partial loss of funds or unintended behavior under certain conditions.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] NLOVault#execute Idle balance is moved without booking deployedAssets, understating NAV and mispricing deposits

Description

The vault's net asset value is totalAssets() = idle depositToken balance + deployedAssets, where deployedAssets is an operator/admin-reported figure. execute() moves idle tokens out of the vault into external positions (swaps / LP) immediately, but deployedAssets is only updated later by a separate reportDeployedAssets() call. During that gap totalAssets() is understated, and because deposit() prices new shares off totalAssets(), anyone depositing in that window mints excess shares and dilutes existing holders.

Vulnerability Details

totalAssets() reads the live idle balance plus the last-reported deployed figure:

```
// NLOVault.sol:554
return depositToken.balanceOf(address(this)) + deployedAssets;
```

execute() forwards operator calldata to a whitelisted target via _forward, which moves the idle depositToken out of the vault. Nothing in execute / _forward increments deployedAssets that is done later, out-of-band, by reportDeployedAssets() (onlyAdmin). So between the execute and the matching reportDeployedAssets, the idle balance has already dropped while deployedAssets is stale, and totalAssets() is understated by the deployed amount.

_depositFor prices shares against that understated value:// NLOVault.sol:756-767

```
uint256 balBefore = depositToken.balanceOf(address(this));
depositToken.safeTransferFrom(payer, address(this), amount);
uint256 received = depositToken.balanceOf(address(this)) - balBefore;
```

```
...
uint256 sharesToMint = (received * supply) / (nav - received); // nav = totalAssets()

With nav understated, nav - received is smaller, so sharesToMint is inflated  the
depositor receives more shares than their contribution is worth, at the expense of
existing holders.
```

Proof of Concept

An illustrative test that shows a deposit in the un-booked window minting inflated shares:

```
function test_ExecuteUnderstatesNAV_MispricesDeposit() public {
    // Alice is the first depositor: 1_000e6
    vm.prank(alice);
    vault.deposit(1_000e6, 0);
    uint256 aliceShares = vault.balanceOf(alice);

    // Operator deploys 900e6 of idle into an external position via execute().
    // Idle balance drops 1000e6 -> 100e6, but deployedAssets is NOT booked yet.
    vm.prank(operator);
    vault.execute(address(dexRouter), _swapCalldata(900e6));
    // deployedAssets still 0  =>  totalAssets() now reads ~100e6 instead of ~1000e6

    // Bob deposits 100e6 during the un-booked window.
    vm.prank(bob);
    vault.deposit(100e6, 0);
    uint256 bobShares = vault.balanceOf(bob);

    // Bob contributed 1/10th of Alice's stake, but priced against the understated
    // NAV he is credited as if he owned roughly half the vault.
    assertGt(bobShares, aliceShares / 2); // correct value would be ~aliceShares/10
}
```

Impact

Existing depositors are diluted: a depositor in the un-booked window captures NAV that belongs to prior holders, and a compromised operator can maximise the window by

deploying nearly all idle capital before self-depositing. Realised at scale, prior depositors lose a material fraction of their NAV per rebalance.

Recommendation

Book the idle↔deployed swing synchronously: adjust deployedAssets by the measured balance delta inside _forward (increase it when idle leaves for a position, decrease it when funds return), so totalAssets() stays continuous across a rebalance. Alternatively, pause deposits for the duration of any execute that moves idle capital.

Status

Resolved

[H-02] NLOIntakeRouter#bridgeBufferedDeposit createdAt is not refreshed on bridge, letting a just-bridged deposit be refunded immediately

Description

A buffered deposit's refund timeout is measured from buffered.createdAt, which is stamped at buffer/record time and never updated when the deposit is bridged. A deposit that sits Buffered for longer than refundTimeout and is then bridged becomes eligible for a source-chain refund the instant it enters Bridging even though the bridge has only just launched and can still deliver on the destination chain. This enables a refund on the source chain plus a delivery on the destination chain for the same deposit (double payment).

Vulnerability Details

createdAt is set at record/receive time and equals the buffer timestamp::

```
// NLOIntakeRouter.sol  _recordBufferedDeposit (L693) / receiveBridgedDeposit (L598)
createdAt: uint64(block.timestamp),
```

The refund gate for a Bridging deposit is measured from that createdAt:

```
// NLOIntakeRouter.sol:323 (claimableRefundAmount)
if (block.timestamp < uint256(buffered.createdAt) + refundTimeout) {
    return 0;
}
```

bridgeBufferedDeposit transitions the deposit to Bridging but does not touch createdAt:

```
// NLOIntakeRouter.sol:489-492
uint256 bridgeAmount = buffered.canonicalAmount;
buffered.canonicalAmount = 0;
buffered.pendingBridgeAmount = bridgeAmount;
buffered.status = BufferedStatus.Bridging;
// (createdAt is never refreshed here)

So the refund timeout is counted from when the deposit was first buffered, not from
when it was bridged. If a deposit is buffered at T0 and bridged at any time after T0
+ refundTimeout, it is immediately refundable while the bridge is still in flight.
```

Proof of Concept

```
function test_CreatedAtStale_RefundsInFlightBridge() public {
    // Deposit buffered at T0
    vm.prank(user);
    (bytes32 id, ) = router.bufferCanonical(1_000e6, user);

    // It sits Buffered longer than the refund timeout
    vm.warp(block.timestamp + router.refundTimeout() + 1);

    // Now it is bridged  createdAt is NOT refreshed
    vm.prank(settlementCaller);
    router.bridgeBufferedDeposit(id, dstChainId, bridgeTarget, fee, bridgeCalldata,
orderHash);

    // The bridge only just launched, yet the deposit is already refundable,
    // because the timeout is measured from the stale buffer-time createdAt.
    assertGt(router.claimableRefundAmount(id), 0);
    // => the source-chain refund can be paid while the destination delivery is
    //    still in flight, producing a double payment for the same deposit.
}
```

Impact

The recorded owner (or a settlement caller) can collect the source-chain refund while the deposit still delivers on the destination chain the protocol pays twice for one deposit, and the shortfall lands on other users' commingled funds.

Recommendation

Refresh the timestamp when the deposit is bridged: set buffered.createdAt = uint64(block.timestamp) inside bridgeBufferedDeposit, so the refund timeout is measured from bridge initiation rather than from the original buffer time. (Consider a dedicated bridgedAt field to keep buffer and bridge timers distinct.)

Status

Resolved

Medium

[M-01] NLOIntakeRouter#restoreBufferedDeposit Partial restore sets status to Buffered while pending remains, corrupting totalPendingBridgeCanonical

Description

restoreBufferedDeposit unconditionally sets the deposit's status back to Buffered, even on a partial restore that leaves pendingBridgeAmount > 0. A subsequent bridgeBufferedDeposit then overwrites pendingBridgeAmount with = while the global totalPendingBridgeCanonical is incremented with +=, permanently over-counting the global pending tally and orphaning the residual.

Vulnerability Details

restoreBufferedDeposit moves an amount back from pending to buffered and always sets Buffered:

```
// NLOIntakeRouter.sol:534-538
buffered.pendingBridgeAmount -= canonicalAmount;
buffered.canonicalAmount     += canonicalAmount;
totalPendingBridgeCanonical  -= canonicalAmount;
totalBufferedCanonical       += canonicalAmount;
buffered.status = BufferedStatus.Buffered; // unconditional, even if pending remains
```

carries pendingBridgeAmount > 0. Re-bridging it overwrites that field:

```
// NLOIntakeRouter.sol:489-494
uint256 bridgeAmount = buffered.canonicalAmount;
buffered.canonicalAmount = 0;
buffered.pendingBridgeAmount = bridgeAmount;      // '=' overwrites the leftover
pending
totalBufferedCanonical      -= bridgeAmount;
totalPendingBridgeCanonical += bridgeAmount;      // '+=' double counts the global
tally
```

The previously-pending residual is dropped from the per-deposit field but was already counted in totalPendingBridgeCanonical, so the global tally is now inflated. The inflated

tally tightens receiveBridgedDeposit's surplus gate (a DoS on future cross-chain credits) and strands the orphaned residual (finalize/restore both require Bridging).

Impact

The global totalPendingBridgeCanonical is permanently over-counted. This tightens receiveBridgedDeposit's surplus gate into a denial-of-service on legitimate future cross-chain credits, and the orphaned residual becomes unrecoverable because both finalize and restore require Bridging status.

Recommendation

Only transition to Buffered when the deposit is fully drained; otherwise keep it Bridging:

```
// NLOIntakeRouter.sol:489-494
buffered.status = (buffered.pendingBridgeAmount == 0)
    ? BufferedStatus.Buffered
    : BufferedStatus.Bridging;
```

Note

Restore is a restricted recovery function and our operational runbook only ever restores a deposit in full, so the partial restore branch that affects the tally is never exercised. The recommended status guard will be included in the next contract iteration.

Status

Acknowledged

[M-02] NLOIntakeRouter#claimableRefundAmount Refund solvency check omits totalPendingBridgeCanonical, allowing a refund against other bridges' backing

Description

The refund solvency check for a timed-out Bridging deposit requires the balance to cover only the buffered deposits plus this one refund it omits the other in-flight bridges (totalPendingBridgeCanonical). When more than one bridge is pending, a refund can be paid out of balance that backs a different in-flight order, so one deposit is refunded on the source chain and still delivered on the destination chain, leaving another user permanently short.

Vulnerability Details

The Bridging-branch guard omits the pending-bridge liabilities:

```
// NLOIntakeRouter.sol:326-334 (claimableRefundAmount)
uint256 refundAmount = buffered.pendingBridgeAmount;
if (refundAmount == 0) { return 0; }
uint256 requiredBalance = totalBufferedCanonical + refundAmount;   // omits
totalPendingBridgeCanonical
if (canonicalToken.balanceOf(address(this)) < requiredBalance) {
    return 0;
}
return refundAmount;
```

The sibling credit path uses the correct, full formula, which is the tell that this one is wrong:

```
// NLOIntakeRouter.sol:586-590 (receiveBridgedDeposit)
require(
    canonicalToken.balanceOf(address(this)) >=
        totalBufferedCanonical + totalPendingBridgeCanonical + canonicalAmount,
    "NLOIntakeRouter: insufficient surplus"
);
```

A safe refund needs the balance to cover all buffered deposits and all other pending bridges (totalPendingBridgeCanonical - refundAmount) plus this refund i.e. totalBufferedCanonical + totalPendingBridgeCanonical. Because the check omits the

other pending bridges, with one slow (still-delivering) and one failed bridge the attacker/owner collects a source-chain refund and the destination delivery for the same deposit, and an honest deposit is left unbacked.

Impact

When two or more bridges are concurrently pending, a timed-out deposit can be refunded on the source chain out of another in-flight bridge's backing while still being delivered on the destination chain a double payment for one deposit that leaves an unrelated user permanently short. It also manifests without malice, from ordinary slow-plus-failed bridge timing.

Recommendation

The correct fix is per-deposit attribution: a refund for deposit X must be payable only from funds that provably returned for X, never from the fungible commingled pool. Bind the refund to the specific bridge order's on-chain cancellation/return a deBridge cancel proof / callback, or a per-intent return-credit call that snapshots the balance delta and records it against X and pay the refund from that attributed amount. With attribution, no aggregate balance check is needed and neither the double-pay nor the DoS occurs. (This is the same root fix as the intake attribution issue and the payout-router accounting in H-01; it also depends on closing "no on-chain proof of bridge delivery/return".)

Status

Resolved

Low

[L-01] NLOVault#setBalancesFromList req.shares rescale is skipped for zero-balance (fully-locked) holders

Description

During a setBalancesFromList rebase, a holder's pending withdrawal-request shares (req.shares) are only rescaled when their current balance is non-zero. A user who requested a full withdrawal has a zero token balance (their shares are held by the vault on their behalf), so the rescale is skipped and their locked request is not adjusted for the rebase desyncing their claim from the new share value.

Vulnerability Details

```
// NLOVault.sol:1480-1485 (setBalancesFromList loop)
if (h != address(this)) {
    WithdrawalRequest storage req = withdrawalRequests[h];
    if (req.shares > 0 && oldBal > 0) {          // oldBal > 0 gate
        req.shares = (req.shares * newBal) / oldBal;
    }
}
```

The comment states the intent: "only when oldBal > 0 without an old reference we cannot derive a fraction." But a fully-locked holder legitimately has oldBal == 0 with req.shares > 0, so their pending request is left un-rescaled while every other holder's balance is rebased over/under-paying that holder on claim once withdrawals are enabled.

Impact

A fully-locked holder's pending withdrawal request is not rebased with the rest of the supply, so on claim (once withdrawals are re-enabled) that holder is over- or under-paid relative to the rebase a per-user locked-share accounting error that silently corrupts their redemption.

Recommendation

Rescale locked requests even when the current balance is zero by using a stable reference (e.g. rescale req.shares by the same rebase factor applied to address(this)'s locked-share balance, or recompute the locked total from withdrawalRequests directly rather than deriving a fraction from oldBal).

Note

Every redemption is reconciled against the holder's request before settlement, so a stale request cannot cause an over or under payment. The rescale logic will be reworked as part of the direct on chain withdrawal release.

Status

Acknowledged

[L-02] NLOVault#execute No cap on how much of the vault balance can be deployed

Description

execute (and the swap/mint paths it validates) can move the entire idle balance into external positions in a single call there is no maximum-deploy percentage or minimum-withdrawal-buffer enforced on-chain. A compromised operator can deploy 100% of idle, leaving no liquidity for pending or near-term withdrawals, and maximising the blast radius of the NAV-lag issue (H-01).

Vulnerability Details

_validateExecuteCall restricts the operator to known DEX selectors and forces recipient == vault, but it never bounds the amount deployed relative to the vault's balance. There is no maxDeployBps, reserved buffer, or per-call size limit. Every idle token is deployable in one transaction.

Impact

A single execute can deploy 100% of idle capital, leaving no buffer for pending or near-term withdrawals (a liveness/DoS on redemptions) and maximising the un-booked-NAV window that H-01 exploits.

Recommendation

Introduce an admin-configured maximum-deploy fraction (e.g. keep at least minBufferBps of totalAssets() idle after any execute that moves capital), and/or a per-call size cap, so a single call cannot strip the withdrawal buffer.

Note

Deployment sizing is enforced by the allocation engine before any execute call is composed, including a per pool TVL share cap and minimum ticket sizes, and execute itself is operator gated to whitelisted targets only. An on chain deployment cap is planned as defense in depth.

Status

Acknowledged

[L-03] NLOVault#payOutAndBurn Payout is sent to an admin-allowlisted recipient, not the share owner

Description

payOutAndBurn transfers the deposit token to an admin-allowlisted recipient while burning sharesToBurn from holderToBurn the fund recipient is decoupled from the share owner. Combined with direct withdrawals being disabled, the actual on-chain destination of a redemption is an admin-controlled address rather than the user whose shares are burned. This is distinct from the "no trustless exit" concern : here the concern is recipient attribution.

Vulnerability Details

```
// NLOVault.sol:928-950 (payOutAndBurn)
function payOutAndBurn(address recipient, uint256 amount, address holderToBurn,
uint256 sharesToBurn)
    external onlyOperator nonReentrant whenNotPaused
{
    if (!allowedPayoutRecipient[recipient]) revert
PayoutRecipientNotAllowed(recipient);
    ...
    _burn(holderToBurn, sharesToBurn);
    depositToken.safeTransfer(recipient, amount);   // funds go to the allowlisted
recipient, not holderToBurn
}
```

recipient and holderToBurn are independent; the burned holder need not (and generally does not) receive the funds. A share owner has no on-chain guarantee that a redemption of their shares is paid to them rather than to an admin-chosen settlement address.

Impact

SA share owners have no on-chain assurance that value for their burned shares reaches them; redemptions are delivered to an admin-controlled address, so users depend entirely on honest off-chain settlement to be paid. Under a compromised operator/recipient this enables paying an admin address while burning a victim's shares.

Recommendation

Where a redemption is meant for the share owner, pay holderToBurn (or a value cryptographically bound to that owner) rather than an arbitrary allowlisted address. If the admin-recipient indirection is required for cross-chain settlement, document the attribution model explicitly and constrain/verify the mapping between holderToBurn and recipient.

Note

This is the intended settlement design: redemptions settle cross chain, so the vault pays into the allowlisted payout infrastructure and the payout router releases funds to the recorded owner of each redemption. The allowlist contains settlement contracts only.

Status

Acknowledged

[L-04] NLOPayoutRouter#emergencyRescue Pays an arbitrary owner-chosen address instead of trying the recorded owner first

Description

emergencyRescue ships a Pending payout to an arbitrary owner-supplied to with no attempt to pay the recorded owner first. The stated purpose is to recover a payout when the owner wallet is unusable, but as written it lets the owner redirect any healthy payout. (It is tracked here specifically for the recipient-attribution fix.)

Vulnerability Details

```
// NLOPayoutRouter.sol:304-320 (emergencyRescue)
function emergencyRescue(bytes32 intentId, address to) external onlyOwner
nonReentrant {
    if (to == address(0)) revert ZeroAddress();
    Payout storage p = payouts[intentId];
    if (p.status != Status.Pending) revert WrongStatus(p.status);
    ...
    p.status = Status.Rescued;
    IERC20(token).safeTransfer(to, amount);   // sends to owner-chosen 'to', never
tries p.owner first
}
```

The recorded p.owner is ignored; funds go wherever to points.

Impact

A malicious or compromised owner can redirect a healthy Pending payout to an arbitrary address instead of the recorded owner, seizing user funds under the guise of an emergency rescue.

Recommendation

Attempt the recorded owner first and fall back only on failure

Note

This is a last resort path for cases where the recorded owner address itself is unusable, for example blocklisted or compromised. It is owner gated and every rescue emits an on

chain event, so any use is publicly visible. In the next version of the payout router the rescue will pay the recorded owner by default, with an alternate destination only where that transfer fails.

Status

Acknowledged

[L-05] NLOIntakeRouter#bufferCanonical / bufferToken Credit the nominal amount instead of the measured received amount (fee-on-transfer over-credit)

Description

The intake router records the caller-supplied amount as the buffered canonical credit without measuring how much it actually received. For a fee-on-transfer or rebasing canonical token, the router receives less than amount but credits the full nominal figure, over-crediting the depositor at the protocol's expense. (The vault's own _depositFor is FoT-safe it measures received so this gap is intake-side only, and latent while the canonical token is USDT.)

Vulnerability Details

```
// NLOIntakeRouter.sol:341-347 (bufferCanonical)
_validateSupportedAmount(address(canonicalToken), amount);
canonicalToken.safeTransferFrom(msg.sender, address(this), amount);
return _recordBufferedDeposit(msg.sender, receiver, address(canonicalToken), amount,
amount);
//                                                          nominal 'amount'
^^^^^^  ^^^^^^ credited as canonical
```

There is no before/after balance measurement; amount is recorded as the canonical credit. bufferToken's canonical branch has the same pattern. Contrast the vault's FoT-safe deposit, which measures the delta.

Impact

For a fee-on-transfer or rebasing canonical token, the router credits more than it actually received; the over-credited surplus is later paid out of the commingled pool, eroding other users' backing. Latent while the canonical token is USDT (no active transfer fee), but a live over-credit the moment a fee-bearing token is used.

Recommendation

Measure the actual received amount and record that.

Note

The canonical settlement token is USDT, which carries no transfer fee, and the supported token list is admin controlled, so no fee bearing token can enter the flow. Measured received accounting will be adopted before any such token is ever listed.

Status

Acknowledged

QA

[Q-01] NLOVault#payOutAndBurn Does not validate that holderToBurn has no pending withdrawal request (req.shares == 0)

Description

payOutAndBurn burns sharesToBurn from holderToBurn without checking whether that holder has an outstanding withdrawal request. Burning shares from a holder whose shares are (partly) committed to a pending request can desync the request accounting.

Vulnerability Details

payOutAndBurn (L928-953) validates the recipient allowlist, non-zero amount/shares, and that holderToBurn holds at least sharesToBurn but it does not read withdrawalRequests[holderToBurn].shares. If the burned holder has a pending request, the request's shares are not reconciled against the burn, leaving stale locked-share accounting.

Impact

Burning shares of a holder with an open withdrawal request leaves the request's shares stale, desyncing locked-share accounting and causing over/under-payment on that request once withdrawals are enabled. No direct loss today (withdrawals disabled); latent correctness bug.

Recommendation

Require withdrawalRequests[holderToBurn].shares == 0 in payOutAndBurn, or reconcile the request (reduce/settle it) as part of the burn, so an operator settlement cannot silently diverge from a pending request.

Note

Informational: holder state is reconciled against any open request before a burn is composed in the settlement flow, so the divergence cannot occur in practice. An on chain guard will be added as part of the direct withdrawal release.

Status

Acknowledged

[Q-02] NLOIntakeRouter#_recordBufferedDeposit Overwrites an existing deposit on a depositId collision (no existence guard)

Description

_recordBufferedDeposit writes the new deposit into bufferedDeposits[depositId] without checking whether that slot is already occupied. If _nextBufferedDepositId ever returns a colliding id, the prior deposit is silently overwritten and its canonicalAmount remains double-counted in totalBufferedCanonical, orphaning the original.

Vulnerability Details

```
// NLOIntakeRouter.sol:687-696 (_recordBufferedDeposit)
depositId = _nextBufferedDepositId(owner, receiver);
bufferedDeposits[depositId] = BufferedDeposit({ ... });   // no existence check
totalBufferedCanonical += canonicalAmount;
```

Contrast receiveBridgedDeposit, which does guard existence:

```
// NLOIntakeRouter.sol:576-579
require(
    existing.owner == address(0) && existing.canonicalAmount == 0 &&
existing.pendingBridgeAmount == 0,
    "NLOIntakeRouter: deposit already exists"
);
```

Impact

A colliding depositId silently overwrites a live deposit: the original is orphaned while its amount stays double-counted in totalBufferedCanonical, corrupting global accounting. Contingent on _nextBufferedDepositId producing a collision, so informational, but the guard is trivial and present in the sibling function.

Recommendation

Add the same existence guard to _recordBufferedDeposit (require the slot is empty before writing), so a colliding id reverts instead of overwriting live state.

Status

Resolved

Centralisation

The NLO Protocol values security and efficiency over decentralisation.

NLO emphasizes a security-focused approach while ensuring operational functionality. By centralizing critical functions, the project enhances reliability and rapid decision-making while maintaining accountability, ensuring the system remains robust and efficient.

Conclusion

After Hashlock's analysis, the NLO project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved or acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au